

Listening to the Philippines (L2Phl)

Operational Manual

Baseline (CAPI) and High-Frequency Panel (CATI):
Implementation, Data Processing, and Quality Control

Avralt-Od Purevjav
Sharon Faye Alariao Piza
Liliana D. Sousa

July 2026

Internal project codename: Project TIPON.

Contents

1	Roles and Responsibilities	4
2	Data Sources and Access	4
3	Sampling	6
4	Questionnaire and Modules	6
5	Fieldwork Operations	7
6	Post-Fieldwork Processing	7
7	Income Imputation (Survey-to-Survey)	8
7.1	Method	8
7.2	GDP adjustment	8
7.3	Validation	9
7.4	Output and link to weighting	9
8	Deliverables and Analytical Outputs	9
8.1	Production and Rebuild Pipeline	9
9	Panel Design	10
10	Questionnaire and Round-Specific Modules	10
11	Per-Round Processing	11
12	Weighting	11
12.1	The target_hh_imputed.dta file	12
13	Pipeline Architecture	13
14	Dashboard Panels	14
15	Maintenance Playbook	14
15.1	Adding a New Round	15
15.2	New or Removed Questions	15
15.3	Updating Filters When Questionnaire Logic Changes	16
16	Verifying Flags and Tracing Numbers	16
16.1	Tracing a Number to Source	16
16.2	Real Issue versus Filtering Artifact	17
16.3	Real-Time Fieldwork Monitoring	17
16.4	Common Causes of Incorrect Flags or Counts	17
16.5	Most Frequent Dashboard Issues	18
17	Conventions and File Maintenance	18
17.1	Naming and the One-Live-File Rule	18
17.2	Files Modified Most Often	18

18 Replication by Stage	19
18.1 Baseline (CAPI)	19
18.2 Panel (CATI) — Per-Round Processing	19
18.3 Quality Control Pipeline	19
18.4 Storylines, Dashboards, and Deep Dives	20
19 Key Conventions	20

Motivation

L2Phl (Listening to the Philippines) is a nationally representative household survey reaching about 108.7 million Filipinos through 2,470 households and 10,496 members across 18 regions. It runs in two modes. A CAPI baseline of 14 modules is administered face-to-face over September and October 2025 and is now frozen; this baseline also serves as the sampling frame. A CATI panel of 10 modules is administered by monthly phone rounds R1 through R9 from November 2025 to July 2026 and rolls forward as new rounds close. The purpose of this manual is to let a new team operate, maintain, and replicate every stage of the workflow. The internal project codename is Project TIPON.

This is the operational manual — it documents how the survey is designed, processed, weighted, and quality-controlled so that a new team can replicate and maintain it. It is not a results report; substantive findings live in the storylines, dashboards, and deep dives cataloged in the Deliverables sections.

Overview

This manual follows the repository layout, moving through the survey lifecycle in the order the folders present it. The CAPI baseline is frozen, while the CATI panel rolls forward with each new round. The QC dashboard and the weighting procedures receive the deepest treatment because they are the live, hand-over-critical workstreams that a new team must be able to run without supervision.

Project Folder Structure

```
L2PHL/
|-- CAPI/
|   |-- Analysis/      (DB dashboard . DD 10 deep dives . SL storyline . SSB)
|   |-- Round00/      (baseline processing: do/, fix/, doc/)
|   |-- S2S/          (survey-to-survey income imputation)
|-- CATI/
|   |-- Analysis/      (QC pipeline . SL storyline . HF masters . do/ . 2share)
|   |-- KOBQ/         (XLSForms R01-R09, skip-logic ground truth)
|   |-- Questionnaire/
|   |-- Field Reports/ (8 Field Documentation Summaries)
|   |-- Round01-Round09/ (per-round processing)
|-- docs/             (specs, plans)
```

Component	Mode	N	Fieldwork	Key design features
Baseline	CAPI (face-to-face)	2,470 HH / 10,496 persons	Sep–Oct 2025	Two-stage stratified PPS; 247 PSUs; 39 strata; sampling frame for CATI
Panel R1–R9	CATI (phone)	~1,100 per round	Nov 2025–Jul 2026	Baseline-household panel; attrition + replenishment; per-round recalibrated weights

Survey at a Glance

Round dates are R1 25 Nov 2025 · R2 28 Dec 2025 · R3 28 Jan 2026 · R4 28 Feb 2026 · R5 30 Mar 2026 · R6 29 Apr 2026 · R7 20 May 2026 · R8 26 Jun 2026 · R9 July 2026. Round 9 fieldwork is in progress at the time of writing (558 households as of mid-July 2026), and its figures should be read as provisional.

Part I: Program Setup

Part I covers the preparatory arrangements that precede data collection: the division of responsibilities between the World Bank team and the implementing firm, and the location of the data and documentation the receiving team works from. Procurement and terms-of-reference history are condensed in this revision; items not recorded in the repository are noted for a future revision.

1. Roles and Responsibilities

The World Bank team is responsible for:

- survey methodology and sampling design;
- questionnaire design and revision;
- construction and verification of survey weights;
- the quality-control (QC) dashboard and monitoring;
- analysis, storylines, and reporting.

The implementing firm is responsible for:

- programming the questionnaires in the survey platform (SurveyCTO/Kobo);
- enumerator recruitment and training;
- conducting CAPI and CATI fieldwork;
- internal quality control;
- delivering raw and cleaned datasets with field documentation.

The terms of reference and vendor-selection history are condensed in this revision; where procurement records are not held in the repository they are noted for a future revision.

2. Data Sources and Access

The receiving team works from four inputs.

The **repository** holds the do-files, the QC pipeline, HTML deliverables, and documentation, all tracked in git; microdata (`.dta`), raw exports, and sample frames are gitignored as sensitive and are not committed. The two mode-level trees are:

```
CAPI/  
CATI/
```

The **Google Drive masters** hold the pooled, harmonized `.dta` datasets, restored to the high-frequency working directory. These files are household-level and confidential, and require World Bank authorization for access:

```
CATI/Analysis/HF/
```

The **Kobo XLSForms** provide the per-round questionnaire definitions and the skip-logic ground truth. One file exists per round, from the Baseline+R1 form through R9:

```
CATI/KOBO/  
CATI/KOBO/L2PHL_CATI_R01.xlsx ... CATI/KOBO/L2PHL_CATI_R09.xlsx
```

The **field reports** comprise eight Field Documentation Summary PDFs, one per collection wave (Baseline+R1 through R8):

```
CATI/Field Reports/
```

Part II: Baseline Household Survey (CAPI)

The CAPI baseline is the first stage of the L2Phl programme and the foundation for everything that follows. Conducted face-to-face between September and October 2025, it produces the nationally representative, household-level estimates that anchor the project, and it serves as the sampling frame from which the CATI panel draws its follow-up respondents. The baseline is frozen once fieldwork closes: its statistics do not change as the panel rolls forward, so it provides a stable reference against which subsequent rounds are measured.

3. Sampling

The baseline uses a two-stage stratified cluster design. The primary sampling unit (PSU) is the barangay, which serves as the enumeration area; the second-stage unit is the household. The realized sample covers 2,470 households and 10,496 members across 247 PSUs.

Strata are defined geographically. Seventeen of the eighteen regions each contribute two strata, an urban stratum and a rural stratum, giving 34 strata. The eighteenth region, the Bangsamoro Autonomous Region in Muslim Mindanao (BARMM), instead enters through five special geographic areas carrying region codes 101 through 105, each its own stratum. Together these give 39 strata in total.

Within this frame, PSUs are selected with probability proportional to size (PPS), and households are then selected within each sampled PSU. This design concentrates fieldwork in a manageable number of enumeration areas while preserving national representativeness after weighting.

Three analytical weights accompany the data, each suited to a different level of analysis. The individual weight scales person-level indicators, so that estimates such as employment, educational attainment, and health status represent the adult or member population. The household weight scales household-level behaviour and decisions, such as finances, asset ownership, and household perceptions. The population weight expresses results in terms of the number of Filipinos affected, supporting welfare and SDG-style framing of the kind "X% of Filipinos live in households where...". The full construction of these weights, including the base weight, the exact measure-of-size correction, post-stratification to census household totals, and age-sex person calibration, is documented in the weighting technical note.

CAPI/Analysis/SL/tex/L2PHL_WEIGHTING_TECHNICAL_NOTE.tex
CAPI/Analysis/SL/tex/L2PHL_WEIGHTING_TECHNICAL_NOTE.pdf

Use the individual weight for person-level indicators (employment, education, health); the household weight for household decisions (finance, assets, perceptions); and the population weight for welfare and SDG framing ("X% of Filipinos live in households where...").

4. Questionnaire and Modules

The baseline instrument comprises 14 modules. Each module has a primary unit of analysis, which determines the weight applied to its indicators.

The module-by-module rationale for each weight assignment follows the weighting technical note cited above, which documents both the construction and the intended use of each weight.

Module	Name	Primary unit / weight
M00	Passport	—
M01	Roster	Individual
M02	Education	Individual
M03	Employment	Individual
M04	Income	Mixed
M05	Finance	Household
M06	Migration	Individual
M07	Health	Individual
M08	Food	Household / individual
M09	Hazards	Household
M10	Dwelling	Household
M11	Sanitation	Household
M12	Utilities	Household
M13	Assets	Household
M14	Views	Household

5. Fieldwork Operations

The CAPI baseline was carried out face-to-face using Tablet-Assisted Personal Interviews on a mobile data-collection platform (SurveyToGo), with the questionnaire developed by the World Bank in collaboration with the Philippine Survey and Research Center (PSRC). Data collection ran from September 18 to October 15, 2025. Each baseline interview lasted approximately two hours, and the call sheet was also programmed in SurveyToGo with embedded GPS coordinates to facilitate field monitoring.

Fieldwork achieved its full target: the completed interviews totalled 2,470 households, distributed across the broad areas as NCR (450), North-Central Luzon (510), South Luzon (530), Visayas (430), and Mindanao (550). A separate pilot of 12 interviews across four language programs (Tagalog, Cebuano, Hiligaynon, and Waray) was conducted before the launch of baseline fieldwork to test the instrument.

The Field Documentation Summary records material operational challenges during the window. Multiple natural disasters struck between September and October 2025, including Super Typhoon Ragasa (September 22), Typhoon Opong (September 26–30), a magnitude 6.9 earthquake offshore near Cebu City (September 30), and a magnitude 7.4 earthquake in Manay, Davao Oriental (October 10). These events disrupted schedules, restricted movement, and limited access to respondents, and were compounded by interviewer attrition (back-outs mid-fieldwork and illness during harsh weather). PSRC responded with prompt interviewer replacement, additional training days, and timeline extensions to secure comprehensive coverage.

Baseline field documentation:

CATI/Field Reports/L2PHL- Field Documentation Summary (Baseline & CATI R1) v2
17December2025.pdf

6. Post-Fieldwork Processing

Raw tablet data is transformed into an analysis-ready sample through a fixed, ordered pipeline of Stata do-files. Each stage reads the output of the previous stage and writes a new dataset, so the full path from raw export to weighted sample is reproducible end to end.

Baseline Processing Pipeline

1. **Stage 1 — stg2raw:** the raw survey export is converted to a Stata .dta file.
2. **Stage 2 — raw2dta:** cleaning, variable renaming, labeling, and consistency corrections are applied.
3. **Stage 3 — psu2wgt:** sampling weights are constructed.
4. **Stage 4 — dta2smp:** the analysis sample is assembled.

Raw data is never modified in place; every cleaning step runs through a Stata do-file so that each transformation can be audited and re-run.

Round00 pipeline do-files:

CAPI/Round00/do/

CAPI/Round00/do/1_L2PHL_CAPI_R00_stg2raw@AP@20260407.do

CAPI/Round00/do/2_L2PHL_CAPI_R00_raw2dta@AP@20260407.do

CAPI/Round00/do/3_L2PHL_CAPI_R00_psu2wgt@AP@20260407.do

CAPI/Round00/do/4_L2PHL_CAPI_R00_dta2smp@AP@20260407.do

7. Income Imputation (Survey-to-Survey)

The L2Phl 2025 baseline does not field a full consumption module, so household welfare cannot be measured directly. Instead it is predicted through survey-to-survey (S2S) imputation from the Family Income and Expenditure Survey–Labor Force Survey (FIES–LFS) 2023, which observes welfare directly. Poverty is then a downstream indicator: predicted welfare is compared against regional poverty lines. The method and diagnostics summarized here follow the technical note prepared by Sandra Segovia Juarez.

7.1 Method

The imputation proceeds in three steps.

S2S imputation

1. **Estimate the welfare model.** A weighted OLS model is fit in FIES–LFS 2023 with log per-capita income as the dependent variable and predictors available in both surveys: demographic composition, household size, assets, housing and sanitation conditions, tenure, urban location, and region fixed effects. The model uses 162,687 observations, with an R-squared of 0.5108 and a root MSE of 0.4798.
2. **Apply the coefficients.** The estimated coefficients are applied to the harmonized L2Phl variables to predict welfare for each household.
3. **Add stochastic residuals.** Resampled source-survey residuals are added to each prediction, so the imputed welfare distribution is not artificially compressed.

Harmonizing the predictor definitions across the two surveys is the critical step, and the most common source of error.

7.2 GDP adjustment

Because the model captures 2023 relationships, predicted welfare is scaled up by cumulative real GDP growth over 2024–2025: $(1 + 0.057) \times (1 + 0.044) = 1.104$. This is implemented as an intercept shift of $\ln(1.104)$, which preserves the ranking of households.

7.3 Validation

Applied back to the 2023 source, the model predicts 15.61 percent poverty against 15.50 percent observed, a close match. In L2Phl 2025 the unadjusted prediction implies 16.96 percent poverty; after the GDP adjustment, the preferred estimate is 13.20 percent.

7.4 Output and link to weighting

The product is `CAPI/S2S/target_hh_imputed.dta`, which holds a per-capita imputed household income measure in the variable `pcinc_imp_mean`. It is copied to `CATI/Analysis/HF/R00/target_hh_imputed.dta` and, in the CATI analysis, merged onto the base-line weight file by household id to attach imputed income for income-quintile breakdowns. It is an imputed-welfare input, not a component of weight construction; Part III documents how it is merged.

Updated 2025 regional poverty lines are not yet available, so the 2025 exercise uses the 2023 regional lines. If those lines rose with inflation, the exercise may modestly understate 2025 poverty.

`CAPI/S2S/S2S_PHL_note.docx` — methodology (Sandra Segovia Juarez).
`CAPI/S2S/s2s_diagnostics.xlsx` — diagnostics. `CAPI/S2S/target_hh_imputed.dta` — imputed output.

8. Deliverables and Analytical Outputs

The baseline findings are disseminated through a set of interactive HTML products; this manual catalogs them and documents how they are rebuilt, but does not reproduce their findings.

Deliverable	Type	Location
16-tab CAPI dashboard	dashboard	<code>CAPI/Analysis/DB/html/l2phl_capi_dashboard.html</code>
12-chapter base-line storyline	storyline	<code>CAPI/Analysis/SL/html/l2phl_baseline_story.html</code>
Wealth deep-dive chapter	storyline	<code>CAPI/Analysis/SL/html/l2phl_wealth_chapter.html</code>
10 thematic deep dives	deep dives	<code>CAPI/Analysis/DD/</code>

The individual deep-dive storylines live under each topic's `html/` subfolder within `CAPI/Analysis/DD/`. These products are published on GitHub Pages at <https://avraltod.github.io/L2PHL/>.

8.1 Production and Rebuild Pipeline

Each deliverable is generated from a Stata do-file that writes results to a machine-readable results file (Markdown or JSON), which a build step then renders into a single self-contained HTML page (Chart.js via CDN, Google Fonts). The storyline-rebuild and replication-audit workflows regenerate a chapter from fresh Stata output, so the reported numbers stay traceable back to the microdata.

Part III: High-Frequency Panel Survey (CATI)

The high-frequency panel is the follow-up phase of the survey. Where the baseline provides a single, once-off portrait of the population, the CATI panel is a monthly telephone survey of the baseline households that tracks fast-moving conditions in the months between rounds. It trades the breadth and depth of the face-to-face baseline for frequency, allowing the programme to monitor how households experience shocks, income changes, and food security as they unfold rather than only at a single point in time.

9. Panel Design

The CATI sample follows the baseline households by phone. Rounds R1 through R9 run monthly from November 2025 to July 2026, giving a rolling monthly reading on the same population that the baseline measured once.

The panel is an *attrition-plus-replenishment* design; it is not a closed cohort. Households drop out across rounds as phone numbers go stale or respondents decline, and new households enter to keep the achieved sample near target. The round-to-round sample is therefore a mix of stayers (households present in consecutive rounds), leavers (households that drop out), and new entrants (households added in later rounds). Any round-over-round comparison must account for this composition rather than assume the same households throughout.

The pooled, harmonized masters that the analysis and quality-control pipeline read live in the high-frequency working directory. Two spine files carry the survey weights, and one pooled file per module carries the harmonized responses across all rounds.

Panel spine masters (person-round and household-round):
 CATI/Analysis/HF/l2phl_cati_individual.dta (holds the individual weight)
 CATI/Analysis/HF/l2phl_cati_household.dta (holds household and population weights)

Per-module pooled masters (M00–M09):
 CATI/Analysis/HF/l2phl_M00_passport.dta
 CATI/Analysis/HF/l2phl_M01_roster.dta
 CATI/Analysis/HF/l2phl_M02_education.dta
 CATI/Analysis/HF/l2phl_M03_shock.dta
 CATI/Analysis/HF/l2phl_M04_employment.dta
 CATI/Analysis/HF/l2phl_M05_income.dta
 CATI/Analysis/HF/l2phl_M06_finance.dta
 CATI/Analysis/HF/l2phl_M07_health.dta
 CATI/Analysis/HF/l2phl_M08_food_nonfood.dta
 CATI/Analysis/HF/l2phl_M09_views.dta

10. Questionnaire and Round-Specific Modules

The CATI instrument has ten modules. It reuses the baseline module structure but is shorter, since a telephone interview cannot carry the full baseline battery.

Module	Name
M00	Passport
M01	Roster
M02	Education
M03	Shocks
M04	Employment
M05	Income
M06	Finance
M07	Health
M08	Food / FIES
M09	Views

Some blocks are round-specific or rotating rather than fixed across all rounds. The health module (M07) carries only a short set of items in most rounds but has a full expansion in Rounds 5 and 8. A Middle-East situation block appears within the Views module (M09) in Rounds 6 through 8. Finance bank and mobile-money gate questions are introduced from Round 5 onward. Because of this evolution, the per-round Kobo XLSForms are the ground truth for exactly which questions exist in each round, and any analysis of a single variable should confirm its presence round by round against the relevant form.

Per-round Kobo XLSForms (ground truth for round-specific content), in CATI/KOBO/:

CATI/KOBO/L2PHL_CATI_R01.xlsx
 CATI/KOBO/L2PHL_CATI_R02.xlsx
 CATI/KOBO/L2PHL_CATI_R03.xlsx
 CATI/KOBO/L2PHL_CATI_R04.xlsx
 CATI/KOBO/L2PHL_CATI_R05.xlsx
 CATI/KOBO/L2PHL_CATI_R06.xlsx
 CATI/KOBO/L2PHL_CATI_R07.xlsx
 CATI/KOBO/L2PHL_CATI_R08.xlsx
 CATI/KOBO/L2PHL_CATI_R09.xlsx

11. Per-Round Processing

Each round is processed from its raw Kobo export through a round-specific do-file and then pooled into the harmonized high-frequency masters that the analysis and quality-control pipeline read. The round do-file cleans and standardizes that round's delivery; the pooling step stacks the cleaned rounds into one file per module, keyed by round and household, so that every downstream analysis reads a single harmonized panel rather than nine separate deliveries.

When the firm delivers revised round data, the masters are re-pooled rather than merely appended. The pooling step rebuilds the harmonized files across all rounds from the current round deliveries, so a late correction to an earlier round is reflected in the masters instead of being lost behind an already-frozen pool. This is why a revised delivery can change values in earlier rounds and not only add the newest round.

12. Weighting

The panel weights are recomputed each round rather than carried forward unchanged, because the achieved sample changes composition from round to round. The construction proceeds in three steps, summarized here; the full derivation is in the weighting technical note cited below.

The individual weight is recalibrated each round to census population cells defined by age group $\times$ sex $\times$ stratum. It starts from a baseline individual weight and is scaled within each cell so that the sum of individual weights in the cell matches the census population total for that cell in that round. The result is the per-round individual weight `indw`.

The population weight for a household is the sum of the individual weights of its members. The household weight is the mean individual weight within the household; it is then smoothed at the primary-sampling-unit level (replaced by the PSU mean) and rescaled so that household totals match census household totals per stratum. These give `popw` and `hhw` on the household-level file.

Full derivation of the panel weights:

`CAP/Analysis/SL/tex/L2PHL_WEIGHTING_TECHNICAL_NOTE.pdf`

12.1 The `target_hh_imputed.dta` file

A recurring firm-turnover question is what `target_hh_imputed.dta` is and whether it is part of the weight construction. It is not. This file is the survey-to-survey (S2S) income-imputation product described in Part II (Income Imputation); it is not produced by the weighting code.

The file is created under `CAP/S2S/target_hh_imputed.dta` and copied to `CATI/Analysis/HF/R00/target_hh_imputed.dta`. In the CATI analysis do-file it is merged onto the baseline weight file by household id: an `m:1` merge on `hhid` that keeps the single variable `pcinc_imp_mean` (labeled “PC HH imputed income”), with `assert(3)` requiring a full match on both sides. The merge attaches a per-capita imputed household income to each household.

The purpose is to enable income-quintile and welfare breakdowns of the panel indicators. `pcinc_imp_mean` is an imputed-welfare input, not a component of the survey weights. This is why it appears in the analysis do-file but not in the earlier weighting do-file: it is joined to the already-weighted file, not used to build the weights.

The merge (analysis do-file) and the baseline weight file it merges onto:

`CATI/Analysis/do/hf_l2phl_analysis@AP020260709.do`

`CATI/Analysis/HF/R00/final_weights.dta`

The imputation method (S2S note):

`CAP/S2S/S2S_PHL_note.docx`

Part IV: Quality Control and the DQ Dashboard

Quality control is the live, hand-over-critical workstream of the panel. Each round the firm delivers new data, and the survey team must confirm that the delivery holds together before any indicator is trusted. The data-quality (DQ) dashboard is the instrument for that confirmation: it is regenerated from the pooled masters by a single command, so the checks never drift from the code. This part documents how to run the pipeline, how to read the panels it produces, how to maintain it as rounds and questionnaires change, and why its outputs can be trusted.

13. Pipeline Architecture

One entry point regenerates everything. The pipeline script `CATI/Analysis/QC/update_pipeline.py`, run as `python3 update_pipeline.py --all`, reads the pooled high-frequency masters, runs the check and build scripts in order, writes intermediate JSON to `CATI/Analysis/QC/cache/`, and produces the HTML dashboard and the Excel cross-round report. Because the checks read a round column in the pooled data, adding a round requires only a few round-count constant edits and registering the new questionnaire form, not changes to the check logic; the step-by-step procedure is in the maintenance playbook.

Full pipeline run

1. **Detect files:** scan `CATI/Analysis/HF/` for pooled and per-round `.dta` masters, `CATI/Questionnaire/` for questionnaire Excel files, and `CATI/Round*/do/` for processing do-files.
2. **Build DQ data (`build_dq.py`):** run the data-quality checks on the pooled masters and write `CATI/Analysis/QC/cache/dq_data.json`.
3. **Build panel tracker (`build_panel.py`):** compute per-round household participation and attrition into `CATI/Analysis/QC/cache/panel_data.json`.
4. **Build interviewer performance (`build_interviewer.py`):** summarize operator performance into `CATI/Analysis/QC/cache/interviewer_data.json`.
5. **Parse questionnaires and Kobo skip logic (`parse_kobo.py`):** extract the cross-round question tables and the Kobo relevant/skip rules per variable into `CATI/Analysis/QC/cache/kobo_skip_logic.json`.
6. **Build issue intelligence (`build_issues.py`):** assemble the tracked data-quality issues into `CATI/Analysis/QC/cache/issues.json`.
7. **Render dashboard (`gen_dashboard.py`):** read the cache and write `CATI/Analysis/QC/output/l2phl_dq_dashboard.html`.
8. **Build Excel report (`build_report.py`):** write the cross-round questionnaire comparison to `CATI/Analysis/QC/output/L2PHL_Questionnaire_Cross_Round_Report.xlsx`.

Single command: `python3 update_pipeline.py --all`

The seven build and check scripts each own one stage of the run:

Script	Role
<code>build_dq.py</code>	Runs the data-quality checks on the pooled masters.
<code>build_panel.py</code>	Builds the household participation and attrition tracker.
<code>build_interviewer.py</code>	Summarizes interviewer/operator performance.
<code>parse_kobo.py</code>	Extracts skip logic from the Kobo XLSForms.
<code>build_issues.py</code>	Assembles the tracked data-quality issues.
<code>gen_dashboard.py</code>	Renders the HTML dashboard from the cache.
<code>build_report.py</code>	Builds the Excel cross-round questionnaire report.

Pipeline entry point and scripts:

CATI/Analysis/QC/update_pipeline.py

CATI/Analysis/QC/scripts/

Generated outputs:

CATI/Analysis/QC/output/l2phl_dq_dashboard.html

CATI/Analysis/QC/output/L2PHL_Questionnaire_Cross_Round_Report.xlsx

14. Dashboard Panels

The dashboard collects, in one page, the panels a reviewer needs to judge a delivery:

- **Completion tracker:** per-round household participation, showing how many baseline households answered each round.
- **Data-quality heatmap:** per-module red/amber/green cells by variable, flagging skip-logic violations, missing mandatory fields, and out-of-range values.
- **Kobo skip-logic panel:** the relevant/skip rules parsed from the XLSForms for each variable, so a flagged cell can be checked against the rule that should govern it.
- **Issue intelligence:** the tracked data-quality issues, with their classification and status.
- **Interviewer performance:** operator-level summaries of throughput and quality.
- **Cross-round questionnaire report:** which variables are present in each round, with new, dropped, and changed questions surfaced across the panel.

15. Maintenance Playbook

Three operations account for nearly all routine upkeep of the data-quality pipeline: adding a new panel round, handling a question that is added to or removed from the questionnaire, and updating the dashboard filters when a skip rule changes. Each is a short, ordered procedure.

15.1 Adding a New Round

Add a round (worked example: R9)

1. Bump the round-count constant in four scripts. Add the new round number and label to the `ROUNDS` list in `build_dq.py`, `build_panel.py`, `build_interviewer.py`, and `build_report.py`. In `build_report.py` also add the round's date label.
2. In `gen_dashboard.py` and `issue_rollup.py`, change the round range `range(1, N)` to `range(1, N+1)`. In `gen_dashboard.py` also extend the JavaScript round arrays, the round labels (`RLABELS`), the CSV export header and rows, and the Module-00 skip-logic fallback round list.
3. Register the round's questionnaire. Add the round's Kobo XLSForm to `KOBO_FILES` in `parse_kobo.py`, and add the round to `ROUND_MAP` in `update_pipeline.py`.
4. Regenerate everything: `python3 update_pipeline.py --all`.
5. Verify. Confirm the new round appears in the rounds list of `CATI/Analysis/QC/cache/dq_data.json`, and that the round's column and label render in `CATI/Analysis/QC/output/l2phl_dq_dashboard.html`.

The pooled `.dta` masters are data-driven by a `round` column, so the checks themselves need no change: only the round-count constants and the new questionnaire form are touched. A partial, in-progress round shows as provisional. Round 9, for example, held 558 households at hand-over; its completeness flags should be read as “round not finished,” not as data problems.

15.2 New or Removed Questions

Handle a new or removed question

1. Check the round's Kobo XLSForm first. It is the ground truth for which variables exist.
2. Update the module's authoritative variable list and heatmap rows in `build_dq.py`: add or remove the variable, and set its expected non-missing condition.
3. In `gen_dashboard.py`, standardize the variable in the module-standardization block so the Tracker, Kobo, and Heatmap panels show it consistently.
4. Annotate variables that are gated by an upstream preload or skip, so they are not falsely flagged as missing.
5. Re-run `python3 update_pipeline.py --all`.

15.3 Updating Filters When Questionnaire Logic Changes

Update dashboard filters

1. Add the new round's Kobo XLSForm to `parse_kobo.py`, as in “Adding a New Round.”
2. `parse_kobo.py` folds each variable's enclosing repeat or group **relevant** condition into that variable, so a changed skip rule flows into the dashboard's skip-logic panel automatically once the new form is registered.
3. Re-run `python3 update_pipeline.py --all`. The refreshed skip-logic panel and gated checks reflect the new logic.

Because the checks are gated by the Kobo **relevant** conditions, updating a filter is usually just registering the new form, not editing check code by hand.

16. Verifying Flags and Tracing Numbers

Two questions come up on every hand-over: where did a given number come from, and is a red cell a real problem or an artifact of how the data were routed. This section gives the tracing procedure and the rules for telling a genuine flag from a filtering artifact.

16.1 Tracing a Number to Source

Every number the dashboard shows traces back through the pipeline to a variable in a pooled `.dta` master and the check that produced it. The chain is short and always the same.

Trace a number

1. **Identify the panel and variable.** Note which dashboard panel the number sits on (Tracker, Kobo, Heatmap, Panel, or Issue) and the variable it reports.
2. **Find it in the cache.** Open the relevant JSON in `CATI/Analysis/QC/cache/` — for the data-quality panels this is `dq_data.json` — and locate the module and variable. The cache holds the computed counts the dashboard renders.
3. **Find the check.** Open `build_dq.py` and locate the check for that module and variable: which authoritative variable list it belongs to and the expected non-missing condition that defines a pass or a flag.
4. **Trace to the data.** Follow the variable to its column in the pooled master in `CATI/Analysis/HF/` (the module master, e.g. the employment or income file, or the combined individual and household files). The check reads that column by round.
5. **Confirm computed statistics against Stata.** If the number is a computed statistic rather than a completeness count, confirm it against the Stata do-file and its log, which are the authoritative source for the figure.

The general L2Phl rule is that Stata do-files are the sole authoritative source for statistics: a number is traced through do-file → log → Excel → HTML, and any divergence is diagnosed at the earliest link before anything downstream is changed. A number that appears in a deliverable but cannot be traced to any do-file or log is treated as an error, not as data. A **cross-checker** helper automates walking a statistic through that chain and reporting where the values diverge.

16.2 Real Issue versus Filtering Artifact

A red cell can be a genuine data problem or a skip-gate mismatch — a variable that is legitimately empty because the respondent was routed past the question. The pipeline gates each check by the Kobo **relevant** condition and annotates variables that are gated by an upstream preload, so the first step on any surprising flag is to check it against the skip logic before treating it as a data error. In practice, several flags that first read as data errors were shown to meet the questionnaire's full routing once the gate was widened.

The issue-intelligence backend classifies each flag by root cause and owner. The categories, defined in `issue_classifier.py` and `issue_model.py`, are:

- **A1 — gate missing.** A constraint is violated and the Kobo skip logic for the variable is itself missing or empty; the questionnaire routing is the likely cause (owner: firm questionnaire).
- **A2 — gate correct, response violates it.** The skip logic is present and correct, but a response breaks it; a genuine field-data problem (owner: firm field).
- **B — gate reference absent.** The questionnaire says the question was asked, but a variable the gate references is absent from the pooled data and no do-file touches it (owner: firm do-file / processing).
- **C — our check disagrees with Kobo.** The check's declared gate disagrees with the Kobo **relevant** condition; a representation issue on our side (owner: us). This rule is held dormant because a variable-set difference cannot distinguish an OR-alternative from a missed conjunct; the gate references are still shown for manual inspection.
- **D — structural, expected.** Not a real issue: derived variables, other-specify (`_oth`) fields, preload-gated variables, or variables not present in the Kobo form for that round (owner: expected).
- **REVIEW — unassigned.** No rule fired; the flag awaits manual judgment.

16.3 Real-Time Fieldwork Monitoring

The dashboard doubles as a live fieldwork monitor. Re-running `python3 update_pipeline.py --all` on the latest pooled masters refreshes completion counts and flags mid-round, so the survey team can watch a round fill in as the firm delivers data. A partial round shows provisional counts: its low completeness is a statement that the round is not finished, not a data problem.

16.4 Common Causes of Incorrect Flags or Counts

Most spurious flags trace to one of a small set of causes:

- **Cross-module variables absent from the pooled .dta.** If a variable a gate depends on is not in the master, the gate cannot be applied and the check may over- or under-count.
- **Preload or pre-filled variables gated upstream.** Variables carried in from a preload are populated by routing, not by a fresh response, and read as anomalies unless annotated.
- **Module items introduced mid-panel.** Items added in a later round — for example, the later FIES items — read as “missing” for the rounds before their introduction.
- **Case-normalized variable names.** Variables whose names differ only in case across rounds must be normalized, or the same item is counted as two.
- **Derived or other-specify variables.** Computed variables and `_oth` fields are excluded from completeness checks; including them produces false flags.
- **Skip-gate drift.** When a round tightens or loosens a routing condition, a gate calibrated to the old logic misclassifies the new data until it is updated.

16.5 Most Frequent Dashboard Issues

The recurring operational issues, in rough order of frequency, are:

- A new round’s questionnaire form not yet registered in `parse_kobo.py`, so its skip logic is not folded into the checks.
- A variable renamed in a later round, which breaks the match against the earlier column.
- A partial, in-progress round read as attrition rather than as an unfinished delivery.
- Stale cache from not re-running the full pipeline after new data or a new form.

17. Conventions and File Maintenance

17.1 Naming and the One-Live-File Rule

Do-files and dated deliverables follow `L2PHL_<MODE>@<ROUND>@AP@YYYYMMDD`. The prefix is always L2PHL (never L2PH) and the author is always AP; the date is the revision date. Exactly one live file exists per slot — a slot being a logical role such as a round master do-file, an analysis script, a stats JSON, or a deliverable — and the live file is the latest-dated `@AP@` version. When the date is bumped, the prior file moves to a `_attic/` folder local to its working directory. `_attic/` is the only archive name in use.

17.2 Files Modified Most Often

Routine pipeline work touches a short list of scripts:

- `build_dq.py` — the checks: authoritative variable lists, expected non-missing conditions, and gates.
- `gen_dashboard.py` — the rendering: module standardization and the panels the dashboard shows.
- `parse_kobo.py` — the skip logic: registering each round’s Kobo form and folding its relevant conditions into each variable.
- `update_pipeline.py` — round registration and orchestration: the single entry point that regenerates everything.

Annex A: Replicability Package

This annex lists the key scripts and folders by workflow stage. All paths are relative to the repository root. Running each stage in sequence reproduces the processing, weights, quality control, and deliverables of the survey: the baseline (CAPI) pipeline, the per-round panel (CATI) processing, the quality control pipeline, and the storyline, dashboard, and deep-dive products.

18. Replication by Stage

18.1 Baseline (CAPI)

The baseline is processed by four numbered do-files in CAPI/Round00/do/, run in order. Each stage reads the output of the prior stage; inputs only ever flow forward.

Stage	Script	Purpose
1	1_L2PHL_CAPI_R00_stg2raw@AP@20260407.do	Assemble the field-server staging exports into a raw dataset.
2	2_L2PHL_CAPI_R00_raw2dta@AP@20260407.do	Clean and label the raw data into module datasets.
3	3_L2PHL_CAPI_R00_psu2wgt@AP@20260407.do	Build the sampling design and compute the survey weights (indw, hhw).
4	4_L2PHL_CAPI_R00_dta2smp@AP@20260407.do	Produce the analysis-ready sample datasets.

The Survey-to-Survey (S2S) income imputation is a separate baseline product; its scripts and datasets live in CAPI/S2S/ and link imputed income to the baseline records for the income-quintile and region breakdowns.

18.2 Panel (CATI) — Per-Round Processing

Each round is processed in its own round folder, one per delivery, from CATI/Round01/ through CATI/Round09/. A round folder holds that round's raw delivery, processing do-file, and fixes. The processed rounds are then pooled into the high-frequency masters and analyzed centrally.

Folder	Contents
CATI/Round01/ ... CATI/Round09/	Per-round processing: raw delivery, round do-file, and fixes.
CATI/Analysis/HF/	Pooled high-frequency panel masters (individual and household), rebuilt when a round is added or re-pooled.
CATI/Analysis/do/	Central panel analysis do-file (hf_l2phl_analysis@AP@20260709.do) run over the pooled masters.

18.3 Quality Control Pipeline

One entry point regenerates the data-quality outputs. The orchestrator CATI/Analysis/QC/update_pipeline.py runs the build and check scripts in CATI/Analysis/QC/scripts/ against the pooled masters and writes the dashboard and cross-round report to CATI/Analysis/QC/output/.

Script	Role
<code>build_dq.py</code>	Runs the data-quality checks on the pooled masters.
<code>build_panel.py</code>	Builds the household participation and attrition tracker.
<code>build_interviewer.py</code>	Summarizes interviewer/operator performance.
<code>parse_kobo.py</code>	Extracts skip logic from the Kobo XLSForms.
<code>build_issues.py</code>	Assembles the tracked data-quality issues.
<code>gen_dashboard.py</code>	Renders the HTML dashboard from the cache.
<code>build_report.py</code>	Builds the Excel cross-round questionnaire report.

18.4 Storylines, Dashboards, and Deep Dives

The analytical products read the finished datasets and present the results. The baseline dashboard is built in `CAPI/Analysis/DB/`, the baseline storyline in `CAPI/Analysis/SL/`, and the ten thematic deep dives in `CAPI/Analysis/DD/`. The panel storyline is built under `CATI/Analysis/SL/`. Each folder holds the do-files or code, results, and the self-contained HTML deliverable for its product.

Deliverable workstreams:

`CAPI/Analysis/DB/` — baseline dashboard
`CAPI/Analysis/SL/` — baseline storyline
`CAPI/Analysis/DD/` — ten thematic deep dives
`CATI/Analysis/SL/` — panel storyline

19. Key Conventions

The pipeline follows a small set of conventions so that any stage can be located and re-run without ambiguity:

- **Date-stamped folders and files:** working outputs carry a YYYYMMDD stamp, so the version used in a run is unambiguous.
- **@AP@ do-file naming, one live file per slot:** each logical role has exactly one live file, the latest-dated @AP@ version; superseded files move to a local `_attic/` folder (for example `CAPI/Round00/do/_attic/`).
- **Three-stage flow:** raw → cleaned/weighted → analysis. Inputs only ever flow forward; a later stage never writes back to an earlier one.
- **Forward-slash paths:** all paths are written with forward slashes, relative to the repository root, so they resolve on any platform.
- **Raw data never modified in place:** raw snapshots are preserved for audit; cleaning and fixes are applied downstream, never to the raw delivery.

Annex B: Turnover Questions — Answered

This annex answers, in the firm’s own grouping, each question raised for the turnover meeting, with a short answer and a pointer to the section that covers it in full.

Question	Short answer	See
Quality Checks		
Will we keep access to the QC Flags Dashboard after turnover?	The dashboard is a self-contained HTML regenerated from the repository pipeline; the receiving team runs and owns it going forward, with no external dependency.	Part IV; Annex C
Will our team own creating and maintaining the QC dashboard? Please walk us through it.	Yes. The full generate-and-maintain process — architecture, the one-command run, and the maintenance playbook — is documented in Part IV.	Part IV
Data Processing — Weighting		
Where does <code>target_hh_imputed.dta</code> come from, and what is its purpose in weighting?	It is the survey-to-survey (S2S) income-imputation product (per-capita imputed income), copied to the high-frequency R00 folder and merged onto the baseline weight file by household id for income-quintile breakdowns; it is not a component of weight construction.	Part II (Income Imputation); Part III (Weighting)
Dashboard		
What edits are needed when a new round of data is uploaded, and how do they reach the dashboard?	Bump the round-count constant in four build scripts, register the round’s Kobo form in <code>parse_kobo</code> and <code>update_pipeline</code> , then run the pipeline; the worked Round 9 example is in the playbook.	Part IV (Playbook)
For rounds with new or removed questions, what edits are needed?	Update the module’s authoritative variable list and heatmap in <code>build_dq</code> and the standardization in <code>gen_dashboard</code> ; the Kobo form is the ground truth for what exists in a round.	Part IV (Playbook)
How can we efficiently update dashboard filters when questionnaire logic changes?	Register the new Kobo form; <code>parse_kobo</code> folds each variable’s enclosing relevant condition into the variable automatically, so the skip-logic panel and gated checks update on re-run.	Part IV (Playbook)
What is the full process from data preparation to final dashboard output?	A single command, <code>update_pipeline.py --all</code> , runs the ordered pipeline from the pooled data to the HTML dashboard and the Excel cross-round report.	Part IV (Architecture)
Other		
How can we trace a number on the dashboard back to the source data?	Follow the chain do-file to Stata log to Excel to HTML; on the dashboard, trace the panel value through the cache JSON to the check in <code>build_dq</code> and the underlying <code>.dta</code> variable. A cross-checker helper automates this.	Part IV (Verifying)
How can we verify a flag is a real issue and not just a filtering-condition problem?	Checks are gated by the Kobo relevant condition; compare any surprising flag against the skip logic and preload gating first, using the issue-classification categories.	Part IV (Verifying)
Can the dashboard monitor fieldwork progress and data quality in real time?	Yes; re-running the pipeline on the latest masters refreshes completion counts and flags mid-round, so it doubles as a live fieldwork monitor.	Part IV (Verifying)
Which files should we modify most often?	<code>build_dq.py</code> (checks), <code>gen_dashboard.py</code> (rendering), <code>parse_kobo.py</code> (skip logic), and <code>update_pipeline.py</code> (round registration).	Part IV (Conventions)
Are there coding standards or naming conventions we should follow?	Yes — the <code>@AP@</code> do-file naming with one live file per slot, superseded files archived to <code>_attic</code> , and forward-slash paths.	Part IV (Conventions); Annex A

Question	Short answer	See
What are the most common causes of incorrect flags or counts?	Cross-module variables absent from the pooled data, preload-gated variables, module items introduced mid-panel, and renamed or derived variables — most red cells trace to one of these rather than to a genuine data error.	Part IV (Verifying)
What are the most frequent dashboard issues?	A new round's questionnaire form not yet registered, a variable renamed in a later round, a partial in-progress round read as attrition, and stale cache from not re-running the full pipeline.	Part IV (Verifying)

Annex C: Data Access and Security

Replication of the workflow requires World Bank authorization for the underlying household-level datasets. The survey data contain confidential household information and are not part of the shared repository: the microdata `.dta` files, the raw exports, and the sample frames are all gitignored and travel outside the version-controlled paths.

Sensitive materials — direct identifiers, phone numbers, and GPS coordinates — are kept out of the shared and replicability paths, following the World Bank Poverty and Equity data-protection protocol. The HTML dashboards and storylines are self-contained single files and are safe to share; the pooled `.dta` masters are not, and should remain within the authorized environment.

To replicate the workflow

1. Obtain access to the repository and to the authorized datasets.
2. Install Stata and the Python dependencies for the QC pipeline.
3. Run each stage in sequence: the baseline Round 00 pipeline, then the per-round CATI processing, then the QC pipeline.
4. Use the procedure boxes in Parts II–IV and the script tables in Annex A as the working reference.